# SPRING BOOT - PRACTICAL EXAM 3

Intermediate | Database Relationships, DTOs, Validation and Custom Queries

| Duration | Total | Pass mark | Prerequisite |
|---|---|---|---|
| 150 minutes | 100 marks | 60% | Exam 2: Database Integration |

Student: _____ Date: _____

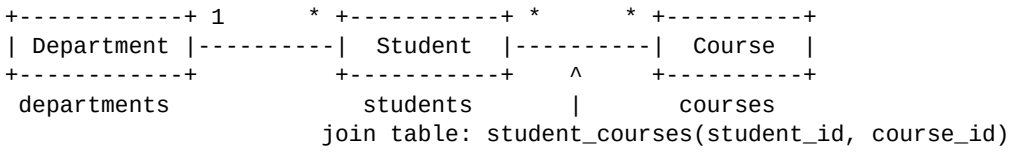
## Scenario and Objective

Continue the Student Management REST API from Exam 2. In a real school a student's course is not just a piece of text: every student **belongs to one department**, and every student can **take many courses** (and every course has many students). In this exam you will model these relationships in MySQL with Spring Data JPA, return clean JSON using DTOs, protect the API with validation and clear error responses, and add search and pagination.

Permitted: Java 17+, Spring Boot 3.x or 4.x (this project uses 4.0.8), Maven, MySQL, Postman or the IntelliJ HTTP Client. You may refer to official documentation. Not required: authentication, frontend UI, Docker.

> **Start with a fresh database.** The `students` table changes shape in this exam (the old `course` text column is removed). `ddl-auto=update` never drops old columns, so if you reuse `student_exam_db` the old NOT NULL `course` column will make every insert fail. Use the new database below.

### Relationship model




## Section A - Setup, Validation and Error Handling (10 marks)

### Question 1: Prepare the project (4 marks)

- Create a MySQL database called `student_relations_db` and point `spring.datasource.url` at it. (2 marks)
- Add `docs/db/create_database.sql` containing the CREATE DATABASE statement. Do not commit a real password. (2 marks)

### Question 2: Validation and consistent error responses (6 marks)

- Re-add Bean Validation so `@Valid` actually works. Invalid input must return **400 Bad Request** with a JSON object mapping each invalid field to a message, e.g. `{"email": "must be a well-formed email address"}`. Use the rules in the table below. (3 marks)
- Create a custom exception (e.g. `ResourceNotFoundException`) and handle it in `GlobalExceptionHandler` so every not-found case returns **404** with the body `{"status": 404, "message": "Student 99 not found"}`. (2 marks)
- Return **409 Conflict** with the same body shape for duplicates and rule violations listed in Section D. (1 mark)

| Request | Field | Rule |
|---|---|---|
| Student | firstName, lastName | Required, not blank |
| Student | email | Required, valid email format |
| Student | age | Required, between 16 and 100 |
| Student | departmentId | Required |
| Department | name | Required, not blank |
| Course | code, title | Required, not blank |
| Course | credits | Required, between 1 and 30 |

## Section B - Entity Relationships (25 marks)

### Question 3: Create the Department entity (6 marks)

| Field | Java type | Database requirement |
|---|---|---|
| id | Long | Primary key, generated automatically |
| name | String | Required, unique (e.g. "Computing") |
| building | String | Optional |
| students | List<Student> | @OneToMany - the inverse side, mapped by Student.department |

- Map the entity to table `departments` and create `DepartmentRepository`. (6 marks total)

### Question 4: Create the Course entity (6 marks)

| Field | Java type | Database requirement |
|---|---|---|
| id | Long | Primary key, generated automatically |
| code | String | Required, unique (e.g. "CS101") |
| title | String | Required |
| credits | Integer | Required |
| students | Set<Student> | @ManyToMany - the inverse side, mapped by Student.courses |

- Map the entity to table `courses` and create `CourseRepository`. (6 marks total)

### Question 5: Update the Student entity (13 marks)

- Remove the old `course` String field. (1 mark)
- Add a `@ManyToOne` `department` field stored in a column called `department_id`. Every student must belong to exactly one department (NOT NULL foreign key). (5 marks)
- Add a `@ManyToMany` `Set<Course> courses` field using a join table called `student_courses` with columns `student_id` and `course_id`. Student is the owning side. (5 marks)
- Start the application and run `SHOW TABLES;` to confirm the four tables exist: departments, courses, students, student_courses. (2 marks)

## Section C - DTOs and JSON Responses (10 marks)

### Question 6: Stop returning entities from controllers (10 marks)

- Create request DTOs (e.g. `StudentRequest` containing `departmentId` instead of a whole Department object) and response DTOs. Controllers must accept and return DTOs, not entities. (4 marks)
- Every endpoint that returns a student must use exactly this shape (4 marks):

```
{
  "id": 1,
  "firstName": "Mary",
  "lastName": "Smith",
  "email": "mary@example.com",
  "age": 22,
  "department": { "id": 1, "name": "Computing" },
  "courses": [ { "id": 2, "code": "DB201", "title": "Databases", "credits": 15 } ]
}
```

- No response may contain endless nesting such as department → students → department → ... (2 marks)

Tip: Java `record` types make short, clean DTOs.

## Section D - Relationship Endpoints (25 marks)

### Question 7: Implement the endpoints below (25 marks)

| HTTP method | Endpoint | Expected behavior | Marks |
|---|---|---|---|
| POST | /api/departments | Create; 201 Created. 409 if the name already exists (ignore case) | 3 |
| GET | /api/departments | List all departments | 1 |
| GET | /api/departments/{id}/students | Students in that department; 404 if the department is missing | 3 |
| DELETE | /api/departments/{id} | 204 on success; 404 if missing; 409 if it still has students | 3 |
| POST | /api/courses | Create; 201 Created. 409 if the code already exists | 2 |
| GET | /api/courses | List all courses | 1 |
| GET | /api/courses/{id}/students | Students enrolled on that course; 404 if the course is missing | 2 |
| POST | /api/students | Body now includes departmentId; 201 with the student JSON. 404 if the department is missing; 409 if the email is already used | 3 |
| POST | /api/students/{id}/courses/{courseId} | Enrol: 200 with updated student JSON. 404 if student or course is missing; 409 if already enrolled | 3 |
| DELETE | /api/students/{id}/courses/{courseId} | Unenrol: 204. 404 if the student is not enrolled. The course itself must not be deleted | 2 |
| GET | /api/students/{id}/courses | Courses for that student; 404 if the student is missing | 2 |

Example POST /api/students request:

```
{
  "firstName": "Mary",
  "lastName": "Smith",
  "email": "mary@example.com",
  "age": 22,
  "departmentId": 1
}
```

**Additional rules.** The Exam 2 endpoints GET/PUT/PATCH/DELETE `/api/students/{id}` must keep working and return the new student JSON. PUT requires `departmentId`; PATCH may include `departmentId` to move a student to another department (other fields unchanged). Changing a student's email to one already used by a *different* student returns 409. Deleting a student must also remove that student's rows from `student_courses`, but never the courses themselves.

## Section E - Custom Queries and Pagination (10 marks)

### Question 8: Query the relationships (10 marks)

- Add a **derived query method** to `StudentRepository` that finds students by last name, ignoring case, and expose it as `GET /api/students/search?lastName=smith`. (3 marks)
- Write your own **JPQL** query using `@Query` that joins students to their courses and returns every student enrolled on a course code. Expose it as `GET /api/students/search?courseCode=DB201`. (4 marks)
- Change `GET /api/students` to support pagination and sorting, e.g. `?page=0&size=2&sort=lastName,asc`. Accept a `Pageable` parameter and return `new PagedModel<>(page)`. The response must contain `content` and `page.totalElements`. (3 marks)

```
{
  "content": [ { "id": 2, "lastName": "Doe", ... }, { "id": 3, "lastName": "Moyo", ... } ],
  "page": { "size": 2, "number": 0, "totalElements": 4, "totalPages": 2 }
}
```

## Section F - Evidence and Verification (10 marks)

> **This section is marked on evidence only.** Working code without the files below earns 0 in this section. Save everything in `docs/exam3_evidence/`.

### Question 9: Run the test script and record the results (10 marks)

Start from an **empty** `student_relations_db` and run these requests in order:

| # | Action | Expected result |
|---|---|---|
| 1 | Create departments Computing (Block A), Business (Block B), Arts | 201 x 3 |
| 2 | Create courses CS101 Intro to Programming (15), DB201 Databases (15), BUS110 Accounting Basics (10) | 201 x 3 |
| 3 | Create students Mary Smith and John Doe (Computing), Tendai Moyo and Ruth Smith (Business) | 201 x 4 |
| 4 | Enrol Mary in CS101 and DB201, John in DB201, Tendai in BUS110 | 200 x 4 |
| 5 | Enrol Mary in CS101 again | 409 |
| 6 | GET students on DB201 | Mary and John |
| 7 | Unenrol Mary from CS101, then repeat the same request | 204, then 404 |

| # | Action | Expected result |
|---|---|---|
| 8 | GET /api/students/search?lastName=smith | Mary and Ruth |
| 9 | GET /api/students?page=0&size=2&sort=lastName,asc | Doe, Moyo; totalElements 4 |
| 10 | PATCH Tendai with only {"departmentId": <Computing id>} | Department is Computing; other fields unchanged |
| 11 | DELETE the Computing department | 409 |
| 12 | POST a student with blank firstName, email "not-an-email", age 12, no departmentId | 400 listing all four fields |
| 13 | DELETE Mary, then GET Mary | 204, then 404 |

- `requests.http` (or an exported Postman collection) containing all 13 steps in order. (2 marks)
- Output of `SHOW CREATE TABLE students;` and `SHOW CREATE TABLE student_courses;` showing the foreign keys. (2 marks)
- Output of `SELECT * FROM student_courses;` after step 4, after step 7 and after step 13. (2 marks)
- Screenshots or saved response text for steps 5, 11, 12 and 13 showing the status code and body. (2 marks)
- An updated `README.md` with setup steps, how to run the app, and a list of all endpoints. (2 marks)

## Section G - Short Theory Questions (10 marks)

### Question 10: Answer all five questions (2 marks each)

Write your answers in `docs/theory_submission/exam3_answers.md`.

1. In a bidirectional relationship, what is the *owning side*, and what does `mappedBy` do? Which side owns the Department-Student relationship in your code?

2. Why can returning JPA entities with bidirectional relationships directly from a controller cause infinite JSON or errors? How do DTOs solve this?

3. Explain `FetchType.LAZY` and `FetchType.EAGER`. What is the default for `@ManyToOne` and for `@ManyToMany`?

4. Why does a many-to-many relationship need a separate join table instead of a column in the students table?

5. Why should deleting a department that still has students return 409 Conflict instead of letting the database error reach the client as a 500?

## Marking Summary

| Section | Marks |
|---|---|
| A - Setup, validation and error handling | 10 |
| B - Entity relationships | 25 |
| C - DTOs and JSON responses | 10 |
| D - Relationship endpoints | 25 |
| E - Custom queries and pagination | 10 |
| F - Evidence and verification | 10 |
| G - Theory | 10 |
| **TOTAL** | **100** |

## Submission Checklist

- Complete Maven Spring Boot source code, committed to your Git branch.
- `docs/db/create_database.sql` and an `application.properties` with no real password.
- Everything listed in Section F inside `docs/exam3_evidence/`.
- Theory answers in `docs/theory_submission/exam3_answers.md`.
- An updated README with setup and run instructions.

**Hints.** Use a `Set` (not a List) for @ManyToMany collections. If you see `LazyInitializationException`, convert entities to DTOs inside a `@Transactional` service method. If @Valid seems to do nothing, check that the annotations are on the class you actually receive in `@RequestBody`.

**Bonus challenge (optional, not graded):** Replace the @ManyToMany with an `Enrollment` entity that also stores `enrolledOn` (date) and `grade`, using two @ManyToOne relationships. Add a @OneToOne `StudentProfile` (phone, address). Write a @SpringBootTest that runs steps 1-7 of the test script automatically.